Project Documentation, Presentation, and Jira Workflows

Software Engineering & Project Management • Student Learning Book

Learning outcomes

• Identify the purpose of common software project documents and keep them useful and current.

• Organize a clear project presentation around the problem, solution, evidence, and lessons learned.

• Explain how Jira can help a team organize requirements, plan work, and track progress.

• Break a requirement into an Epic, Stories, Tasks, and Bugs without losing traceability.

• Create well-written work items with descriptions, acceptance criteria, priority, owner, and status.

• Read a simple workflow board and use it to communicate project progress.

Running case study: A team is delivering a College Event Management System (CEMS). Students can discover events and register; organizers publish events, manage capacity, and review attendance.

1. Why Project Documentation Matters

Project documentation records the information people need to understand, build, test, operate, and maintain a software product. Good documentation reduces misunderstandings, supports handover, provides evidence for decisions, and helps new team members contribute.

Documentation is not valuable merely because it is long. It should have a clear audience, purpose, owner, and update process. Keep it concise enough to use, detailed enough to avoid ambiguity, and aligned with the actual product.

Document / artefact	Purpose	CEMS example
Project charter or overview	States the problem, objective, scope, stakeholders, constraints, and success measures	Reduce manual event registration and provide reliable attendance data
SRS / requirements specification	Describes functional and non-functional requirements and constraints	Students can register only when seats are available
Design / architecture note	Explains major components, interfaces, data, and design decisions	Web UI, application API, database, notification service
Project plan	Records milestones, responsibilities, dependencies, estimates, and risks	Requirements, prototype, build, test, pilot, release
Test plan and test cases	Defines test scope, approach, scenarios, data, and expected outcomes	Capacity limit, duplicate registration, invalid input
User / operations guide	Explains how to use or support the system	Organizer guide for creating and closing an event
Release notes	Summarizes changes, known issues, and upgrade or usage notes	Registration confirmation added; reporting remains planned

2. A Practical Documentation Standard

For each document or page, answer five questions: who is it for, why does it exist, who owns it, where is the current version, and when should it be reviewed?

• **Use stable names:** for example, CEMS-SRS-v1.2 or a clearly titled wiki page.

• **Record decisions:** write the decision, reason, date, and consequences—not just the final choice.

• **Link related items:** connect requirements to design decisions, tests, defects, and releases.

• **Control sensitive information:** do not put passwords, API keys, or private student data in ordinary project documents or Jira comments.

• **Keep status honest:** distinguish proposed, approved, implemented, tested, and released.

• **Review changes:** update documents when the product or process changes materially.

Example decision record: “Decision: prevent overbooking using a server-side capacity check. Reason: a client-side check alone can be bypassed or become stale when two students register simultaneously. Consequence: registration must handle concurrent requests safely.”

3. Preparing a Project Presentation

A strong project presentation tells a coherent story. The audience should understand the problem, the people affected, what the team built, how it works, what evidence supports its quality, and what remains to be done.

Slide	What to include	CEMS example
1. Title and team	Project name, members, course, date	College Event Management System
2. Problem	Current pain points and who experiences them	Manual forms, duplicate entries, uncertain seat counts
3. Objectives and scope	What success means and what is excluded	Browse events and register; advanced analytics excluded
4. Stakeholders and requirements	Main users and important requirements	Students, organizers, administrator
5. Design	Architecture, data model, or UML diagram	Browser, API, database, notification service
6. Implementation	Key features and technology choices	Event listing, registration, capacity check
7. Demonstration	A short, rehearsed end-to-end user journey	Open event, register, show confirmation
8. Testing and evidence	Test cases, results, limitations, known defects	Duplicate-registration and capacity-limit tests
9. Project management	Timeline, work distribution, risks, changes	Sprint board and one risk response
10. Conclusion	What was achieved, lessons, next steps	Pilot with one department; add reports later

Presentation practices

• Use readable text, meaningful diagrams, and consistent terminology.

• Explain why a design choice was made instead of reading every bullet aloud.

• Rehearse the demo with prepared test data and a backup screenshot or recording.

• Show evidence honestly: distinguish completed features from mock-ups or planned work.

• Plan time for questions and ensure each team member can explain their contribution.

4. Introduction to Jira

Jira is a work-management tool commonly used to track requirements, tasks, bugs, and project progress. Teams configure projects and workflows to match how they work. Labels and screen names may differ between Jira editions and project types, so follow the configuration used by your institution or organization.

Jira supports visibility and traceability; it does not replace good communication, sound engineering, or thoughtful prioritization.

Common Jira concepts

Concept	Meaning	Example
Project / space	Container for related work and its configuration; terminology can vary by Jira experience	CEMS project
Issue / work item	A record of a piece of work	Create event registration

Concept	Meaning	Example
Epic	A larger body of work grouping related items	Student Event Registration
Story	A user-focused capability or outcome	As a student, I want to register for an event
Task	Work that needs to be done, often technical or operational	Add registration endpoint
Bug	A defect where actual behavior differs from expected behavior	System allows registration beyond capacity
Backlog	Prioritized work not yet selected or completed	Registration, event browsing, reporting
Board	Visual view of work moving through a workflow	To Do → In Progress → Done
Sprint	A fixed-length Scrum work period	Two-week CEMS sprint
Workflow	Statuses and transitions used to track work	To Do, In Progress, In Review, Done

5. Breaking a Requirement into Jira Work Items

A requirement often needs several work items. The goal is to make the work understandable and trackable without splitting it into so many tiny tickets that the board becomes noise. The hierarchy below is a useful example, not a universal mandatory Jira hierarchy.

> Requirement: Students must be able to register for an event only when seats are available.

Level	Work item	What it captures
Epic	Student Event Registration	A broad capability delivered through related work
Story 1	Student can view event capacity status	User-visible outcome and acceptance criteria
Story 2	Student can register for an available event	User-visible outcome and acceptance criteria
Task	Implement registration API	Technical work supporting the story
Task	Create registration confirmation screen	UI work supporting the story
Task	Add automated capacity tests	Quality work supporting the story
Bug	Registration succeeds after capacity is full	Unexpected behavior with reproduction steps

In Jira, the exact parent-child relationships available depend on project type and configuration. For teaching, focus on the traceability: the Epic groups the outcome, Stories describe user value, Tasks describe implementation work, and Bugs capture defects.

6. Anatomy of a Good Jira Issue

- **Summary:** short, specific title that communicates the outcome or problem.
- **Description:** context, user need, relevant constraints, and links.
- **Acceptance criteria:** observable conditions for completion.
- **Priority:** relative urgency or importance using the team's agreed scale.
- **Assignee:** the person currently responsible for progressing the item, if assignment is used.
- **Status:** where the item sits in the workflow.
- **Estimate:** a team-agreed estimate where estimation is part of the process.
- **Links:** parent, dependency, related requirement, test, or defect where supported.
- **Evidence:** screenshots, test results, or review links when appropriate; never secrets or unnecessary personal data.

Example issue

> **Summary:** Prevent registration when an event has no available seats

Description: A student must not receive a confirmed registration when the event's confirmed registration count has reached its capacity. The capacity check must be performed safely on the server.

Acceptance criteria:

• When at least one seat remains, a valid student can register and receive a confirmation reference.

• When no seats remain, registration is rejected with a clear message.

• Repeated attempts do not create duplicate registrations for the same student and event.

• Automated tests cover the available-seat, full-capacity, and duplicate-registration cases.

7. Workflow and Board Hygiene

A workflow represents the stages through which work moves. A simple workflow might be To Do → In Progress → In Review/Test → Done. Some teams include Blocked, Ready for Testing, or other states.

• Move an item when its actual state changes; do not update the board only before a presentation.

• Agree on what each status means and what evidence is needed to leave it.

• Make blockers visible and communicate dependencies early.

• Keep issue descriptions and acceptance criteria updated when scope changes.

• Use a clear Definition of Done so “Done” means more than “coding finished.”

• Avoid creating duplicate tickets for the same work; link related issues where appropriate.

Important: Jira status alone does not prove that a feature works. The team should use code review, test results, acceptance criteria, and a usable increment as evidence.

8. Classroom Practical: Create a Jira Project Plan

Work in a small team. If Jira access is unavailable, create the same structure in a spreadsheet or on a paper board.

1. Create or identify a project called CEMS (College Event Management System).

2. Write one Epic for Student Event Registration.

3. Add at least three Stories with a user-focused summary and acceptance criteria.

4. Add two implementation Tasks and one Bug; link them to related work where the configuration permits.

5. Set priorities and create a simple workflow board.

6. Choose a Sprint Goal and select a realistic subset of backlog items.

7. Move one item through the workflow and record the evidence required before it is Done.

8. Prepare a five-slide mini-presentation explaining the requirement, work breakdown, board, test evidence, and remaining risk.

Expected submission: screenshot or export of the backlog/board (or equivalent table), one Epic, three Stories, two Tasks, one Bug, acceptance criteria, Sprint Goal, and a short project presentation.

9. Sample Backlog Table

Type	Summary	Priority	Acceptance / completion evidence
Epic	Student Event Registration	High	Related registration work is grouped and traceable
Story	View event details and capacity	High	Title, date, venue, and availability are visible
Story	Register for an available event	High	Confirmation reference is shown after successful registration
Task	Implement server-side capacity check	High	Code review complete and relevant tests pass
Task	Add registration confirmation UI	Medium	UI displays success and failure messages clearly

Type	Summary	Priority	Acceptance / completion evidence
Bug	Full event accepts another registration	Highest	Reproduced, fixed, retested, and regression check passes

10. Review Questions with Suggested Answers

1. Why document a software project?

Suggested answer: To preserve shared understanding, decisions, requirements, plans, quality evidence, and handover information.

2. What makes documentation useful?

Suggested answer: A clear audience and purpose, accurate content, an owner, traceability, and timely updates.

3. What is the purpose of a project presentation?

Suggested answer: To explain the problem, solution, evidence, decisions, outcomes, limitations, and next steps.

4. What is an Epic?

Suggested answer: A larger body of work that can be delivered through multiple related work items.

5. How does a Story differ from a Task?

Suggested answer: A Story usually describes user value; a Task commonly describes implementation or supporting work.

6. What information should a Bug include?

Suggested answer: Expected versus actual behavior, steps to reproduce, environment or context, and evidence where useful.

7. Why are acceptance criteria important?

Suggested answer: They make expected behavior observable and help guide implementation and testing.

8. Does a Jira item marked Done prove quality?

Suggested answer: No. The team's Definition of Done and test/review evidence should support completion.

9. What is a workflow?

Suggested answer: The statuses and transitions used to represent how work progresses.

10. What should a team show in a project demo?

Suggested answer: A rehearsed, representative user journey with honest explanation of what is complete and what remains.

11. Glossary

Term	Meaning
Traceability	Ability to connect related requirements, design, implementation, tests, and defects
Acceptance criteria	Conditions used to judge whether a work item meets its expected behavior
Epic	A larger grouping of related work
Story	A user-oriented capability or outcome
Task	A defined unit of implementation or supporting work
Bug	A defect where observed behavior differs from expected behavior
Workflow	Statuses and transitions that represent work progress
Backlog	Ordered list of work not yet completed or selected
Evidence	Test, review, demonstration, or other information supporting a claim
Release notes	Summary of changes and relevant known issues in a release

12. Homework

1. Prepare a one-page project overview for a Library Management System or Hospital Appointment System.

2. Create a ten-item Jira-style backlog containing one Epic, Stories, Tasks, and at least one Bug.

3. Write acceptance criteria for two Stories and reproduction steps for the Bug.

4. Prepare a five-slide presentation with a problem statement, requirements, design, testing evidence, and future work.

5. Write a short reflection on how linking requirements, issues, and tests improves project traceability.

Next chapter: Day 27 — Behaviour-Based Testing (BDD), Gherkin scenarios, and connecting acceptance criteria to testable behavior.